Atividade de Programação: Serviço de Processamento de Arquivos com gRPC

Disciplina: Computação Distribuída

Profs. Alcides Teixeira Barboza Júnior e Mário O. Menezes

Resumo:
Nesta atividade, vocês serão desafiados a aplicar os conceitos fundamentais de computação distribuída através da construção de um serviço de processamento de arquivos. Utilizando o gRPC, a solução deverá estabelecer uma comunicação eficiente por streaming entre um cliente em Python e um servidor em C++.

O servidor terá a responsabilidade de integrar ferramentas externas do ecossistema Unix (como o Ghostscript, Poppler e ImageMagick) para executar quatro tarefas específicas: compressão de PDFs, extração de texto, conversão de formatos de imagem e redimensionamento. Adicionalmente, o projeto exige a implementação de um sistema robusto de logging para rastrear requisições e falhas, bem como a conteinerização completa da solução utilizando o Docker, culminando na apresentação do código-fonte e execução do projeto em laboratório.

Objetivo Geral:

- Compreender e aplicar os conceitos básicos de gRPC no desenvolvimento de sistemas distribuídos.
- Implementar um serviço gRPC para processamento de arquivos, explorando diferentes tipos de RPC (unário e streaming cliente-servidor).
- Utilizar C++ para o servidor gRPC e Python para o cliente gRPC.
- Integrar ferramentas de linha de comando externas (Ghostscript e ImageMagick) em um serviço gRPC.
- Implementar a solução utilizando containers (Docker)
- Implementar logging para rastreamento e depuração de serviços distribuídos.

Objetivos Específicos:

- Definir e implementar serviços gRPC utilizando Protocol Buffers.
- Criar um servidor gRPC em C++ que ofereça serviços de processamento de arquivos.
- Desenvolver um cliente gRPC em Python que consumam os serviços do servidor.
- Utilizar streaming cliente-servidor para o envio eficiente de arquivos para processamento.
- Implementar funções para compressão de PDF, conversão de PDF para TXT, conversão de formato de imagem e redimensionamento de imagem.
- Implementar o isolamento dos serviços utilizando containers
- Registrar logs de todas as operações realizadas no servidor, incluindo informações de requisição e status de conclusão.
- Tratar erros e retornar status adequados para os clientes gRPC.

Pré-requisitos:

- Conhecimentos básicos de programação em C++ e Python.
- Familiaridade com a linha de comando e sistemas Unix-like (para instalação e uso de Ghostscript e ImageMagick).
- Noções básicas de computação distribuída e RPC.

Entrega:

- Execução em laboratório
- Código fonte completo do servidor gRPC (C++).
- Código fonte completo do cliente gRPC (Python).
- Arquivo protobuf (.proto) definindo os serviços e mensagens gRPC.
- Arquivo de log gerado pelo servidor durante os testes.
- Relatório sucinto descrevendo a implementação, os desafios encontrados e as decisões de design tomadas, e os prints das telas de execução.

Serviços a serem Implementados:

O servidor gRPC deverá oferecer os seguintes serviços de processamento de arquivos:

1. **Compressão de PDF (CompressPDF) (Entrega 1):**
   - **Entrada:** Arquivo PDF (streaming cliente-servidor).
   - **Saída:** Arquivo PDF comprimido (streaming servidor-cliente) ou status de sucesso/falha (unário).
   - **Processamento:** Utilizar o comando gs (Ghostscript) para comprimir o arquivo PDF recebido.
   - **Comando gs Exemplo**: gs -sDEVICE=pdfwrite -dCompatibilityLevel=1.4 -dPDFSETTINGS=/ebook -dNOPAUSE -dQUIET -dBATCH -sOutputFile=output.pdf input.pdf

2. **Conversão de PDF para TXT (ConvertToTXT) (Entrega 2):**
   - **Entrada:** Arquivo PDF (streaming cliente-servidor).
   - **Saída:** Conteúdo do arquivo TXT (streaming servidor-cliente) ou status de sucesso/falha (unário).
   - **Processamento:** Utilizar o comando pdftotext (parte do pacote Poppler, frequentemente instalado com Ghostscript ou separadamente) para converter o PDF para texto.
   - **Comando pdftotext Exemplo:** pdftotext input.pdf output.txt

3. **Conversão de Formato de Imagem (ConvertImageFormat) (Entrega 2):**
   - **Entrada:** Arquivo de imagem (streaming cliente-servidor), formato de saída desejado (string).
   - **Saída:** Arquivo de imagem convertido (streaming servidor-cliente) ou status de sucesso/falha (unário).
   - **Processamento:** Utilizar o comando convert (ImageMagick) para converter o formato da imagem.
   - **Comando convert Exemplo:** convert input.jpg output.png (onde .png é o formato de saída desejado).

4. **Redimensionamento de Imagem (ResizeImage) (Entrega 2):**
   - **Entrada:** Arquivo de imagem (streaming cliente-servidor), largura e altura desejadas (inteiros).
   - **Saída:** Arquivo de imagem redimensionado (streaming servidor-cliente) ou status de sucesso/falha (unário).
   - **Processamento:** Utilizar o comando convert (ImageMagick) para redimensionar a imagem.
   - **Comando convert Exemplo:** convert input.jpg -resize 800x600 output.jpg (onde 800x600 são as dimensões desejadas).

Log de Operações:

- O servidor deve registrar todas as requisições de serviço em um arquivo de log (ex: server.log).
- Cada entrada de log deve incluir: ○ Timestamp da requisição.
    - Nome do serviço requisitado.
    - Nome do arquivo de entrada (se aplicável).
    - Status da operação (sucesso, falha, mensagem de erro em caso de falha).

Passos para Implementação:

1. Definição do Protobuf (.proto):

- Defina as mensagens para requisição e resposta de cada serviço.
- Utilize streaming para envio e recebimento de arquivos.
- Inclua campos para parâmetros específicos de cada serviço (formato de saída, dimensões, etc.).
- Defina o serviço FileProcessorService com os métodos correspondentes a cada serviço (CompressPDF, ConvertToTXT, ConvertImageFormat, ResizeImage).

2. Implementação do Servidor gRPC (C++):

- Crie um servidor gRPC em C++ utilizando a biblioteca gRPC.
- Implemente a classe de serviço FileProcessorServiceImpl que herda da classe base gerada pelo protobuf.
- Implemente cada método de serviço (CompressPDF, ConvertToTXT, ConvertImageFormat, ResizeImage).
    - Receba o arquivo como um stream de bytes do cliente.
    - Salve o arquivo recebido temporariamente no servidor.
    - Execute o comando de linha de comando apropriado (gs, pdftotext, convert) para processar o arquivo.
    - Capture a saída e o código de retorno do comando.
    - Registre a operação no arquivo de log, incluindo status e mensagens de erro se houver.
    - Retorne o arquivo processado como um stream de bytes para o cliente ou um status de sucesso/falha.
    - Limpe os arquivos temporários.
- Configure o servidor para ouvir em uma porta específica.

3. Implementação do Cliente gRPC (Python):

- Crie o cliente gRPC em Python utilizando as bibliotecas gRPC correspondentes.
- Implemente funções no cliente para interagir com cada serviço do servidor.
    - Peça ao usuário para selecionar o arquivo de entrada e fornecer os parâmetros necessários (formato, dimensões, etc.).
    - Abra o arquivo de entrada e envie-o como um stream de bytes para o servidor utilizando o streaming cliente-servidor do gRPC.
    - Receba a resposta do servidor (stream de bytes ou status).
    - Se receber um stream de bytes (arquivo processado), salve-o no disco.
    - Exiba mensagens de sucesso ou erro para o usuário.

4. Testes e Geração de Log:

- Execute o servidor gRPC.
- Execute o cliente gRPC (Python) para testar cada serviço com diferentes arquivos de entrada e parâmetros.
- Verifique o arquivo de log do servidor para garantir que todas as operações foram registradas corretamente.
- Teste cenários de erro (arquivo não encontrado, formato inválido, etc.) e verifique se o servidor e os clientes lidam com os erros adequadamente.

Protobuf (.proto):

```
syntax = "proto3";

// Define o namespace para organizar o código gerado e evitar conflitos no C++ e Python
package file_processor;

// Estrutura que carrega uma fração dos bytes do arquivo durante a transmissão em fluxo (streaming)
message FileChunk {
   bytes content = 1;
}

// Estrutura principal de requisição enviada do cliente para o servidor
message FileRequest {
   string file_name = 1;
   FileChunk file_content = 2;

   // 'oneof' garante que apenas um bloco de parâmetros seja enviado na requisição, economizando banda
   oneof parameters {
      CompressPDFRequest compress_pdf_params = 3;
      ConvertToTXTRequest convert_to_txt_params = 4;
      ConvertImageFormatRequest convert_image_format_params = 5;
      ResizeImageRequest resize_image_params = 6;
   }
}

message CompressPDFRequest {}
message ConvertToTXTRequest {}

message ConvertImageFormatRequest {
   string output_format = 1; // Define a extensão de saída desejada (ex: "png", "jpg")
}

message ResizeImageRequest {
   int32 width = 1;
   int32 height = 2;
}

message FileResponse {
   string file_name = 1; // Nome do arquivo gerado/processado
   FileChunk file_content = 2;
   string status_message = 3;
   bool success = 4;
}

// Contrato de comunicação que o servidor deve implementar e o cliente pode consumir
service FileProcessorService {
   // A palavra-chave 'stream' em ambos os lados indica Streaming Bidirecional.
   // Isso permite processar arquivos pesados (como PDFs e imagens grandes) enviando-os
   // em pequenos blocos (chunks), evitando estouro de memória RAM nos contêineres.
   rpc CompressPDF (stream FileRequest) returns (stream FileResponse);
   rpc ConvertToTXT (stream FileRequest) returns (stream FileResponse);
   rpc ConvertImageFormat (stream FileRequest) returns (stream FileResponse);
```

```
    rpc ResizeImage(stream FileRequest) returns (stream FileResponse);
}
```

Trechos de Código (C++ Servidor):

```cpp
#include <fstream>
#include <iostream>
#include <memory>
#include <string>
#include <chrono>
#include <ctime>
#include <cstdlib>
#include <grpcpp/grpcpp.h>
#include "file_processor.grpc.pb.h"

using grpc::Server;
using grpc::ServerBuilder;
using grpc::ServerContext;
using grpc::ServerReaderWriter;
using grpc::Status;
using namespace file_processor;

class FileProcessorServiceImpl final : public FileProcessorService::Service {
private:
    // ================================================================================
    // IMPLEMENTAÇÃO SUA (MÉTODOS AUXILIARES)
    // ================================================================================

    void LogOperation(const std::string& type, const std::string& service, const
std::string& file, const std::string& msg) {
        // TODO: Implementar a lógica de gravação no arquivo "server.log".
        // 1. Obtenha a data e hora atual do sistema (ex: usando std::chrono e
std::ctime).
        // 2. Abra o arquivo "server.log" em modo de adição (std::ios::app).
        // 3. Formate e escreva a linha de log contendo o timestamp, type, service, file
e msg.
        // 4. Feche o arquivo.
    }

    // ================================================================================
    // MÉTODOS AUXILIARES FORNECIDOS (NÃO ALTERAR)
    // ================================================================================

    bool ReceiveFile(ServerReaderWriter<FileResponse, FileRequest>* stream, std::string&
filepath, std::string& original_name, FileRequest& last_req) {
        FileRequest req;
        std::ofstream outfile;
        bool first = true;
        while (stream->Read(&req)) {
            if (first) {
                original_name = req.file_name();
                filepath = "/tmp/input_" + original_name;
                outfile.open(filepath, std::ios::binary);
                last_req = req;
                first = false;
            }
```

```
            if(req.has_file_content()) {
                outfile.write(req.file_content().content().c_str(),
req.file_content().content().size());
            }
        }
        if (outfile.is_open()) outfile.close();
        return !first;
    }

    Status SendFile(ServerReaderWriter<FileResponse, FileRequest>* stream, const
std::string& out_filepath, const std::string& out_filename) {
        std::ifstream infile(out_filepath, std::ios::binary);
        if (!infile) return Status(grpc::StatusCode::INTERNAL, "Falha ao ler output");

        FileResponse resp;
        resp.set_success(true);
        resp.set_file_name(out_filename);
        resp.set_status_message("Sucesso");

        char buffer[1024];
        while (infile.read(buffer, sizeof(buffer)) || infile.gcount() > 0) {
            FileChunk* chunk = new FileChunk();
            chunk->set_content(buffer, infile.gcount());
            resp.set_allocated_file_content(chunk);
            stream->Write(resp);
            resp.clear_file_content();
        }
        return Status::OK;
    }

public:
    // ================================================================================
    // IMPLEMENTAÇÃO SUA (SERVIÇOS gRPC)
    // ================================================================================

    Status CompressPDF(ServerContext* context, ServerReaderWriter<FileResponse,
FileRequest>* stream) override {
        // TODO: Implementar a lógica de compressão de PDF aqui.
        // 1. Use ReceiveFile() para ler o stream de dados vindo do cliente.
        // 2. Formule o comando do Ghostscript (gs) para comprimir o PDF.
        // 3. Execute o comando com std::system().
        // 4. Registre o resultado usando a sua função LogOperation().
        // 5. Use SendFile() para devolver o arquivo comprimido ao cliente.
        // 6. Remova os arquivos temporários gerados na pasta /tmp.

        return Status::OK;
    }

    // ================================================================================
    // MÉTODOS IGNORADOS PARA ESTA ENTREGA
    // ================================================================================

    Status ConvertToTXT(ServerContext* /*context*/, ServerReaderWriter<FileResponse,
FileRequest>* /*stream*/) override {
        // Não é necessário implementar para esta atividade.
```

```
        return Status::OK;
    }

    Status ConvertImageFormat(ServerContext* /*context*/,
ServerReaderWriter<FileResponse, FileRequest>* /*stream*/) override {
        // Não é necessário implementar para esta atividade.
        return Status::OK;
    }

    Status ResizeImage(ServerContext* /*context*/, ServerReaderWriter<FileResponse,
FileRequest>* /*stream*/) override {
        // Não é necessário implementar para esta atividade.
        return Status::OK;
    }
};

void RunServer() {
    std::string server_address("0.0.0.0:50051");
    FileProcessorServiceImpl service;

    ServerBuilder builder;
    builder.AddListeningPort(server_address, grpc::InsecureServerCredentials());
    builder.RegisterService(&service);

    std::unique_ptr<Server> server(builder.BuildAndStart());
    std::cout << "Servidor gRPC operacional em " << server_address << std::endl;
    server->Wait();
}

int main() {
    RunServer();
    return 0;
}
```

Trecho de Código de Exemplo (Python Cliente):

```
import os
import grpc
import file_processor_pb2
import file_processor_pb2_grpc

# ================================================================================
# CONFIGURAÇÃO DE REDE (NÃO ALTERAR)
# Desliga a interferência do proxy do Codespaces na rede interna do Docker
# ================================================================================
os.environ['http_proxy'] = ''
os.environ['https_proxy'] = ''
os.environ['no_proxy'] = '*'

# ================================================================================
# MÉTODOS AUXILIARES FORNECIDOS (NÃO ALTERAR)
# ================================================================================

def generate_requests(file_path, request_params):
    file_name = os.path.basename(file_path)
    # Envia os parâmetros na primeira mensagem
    yield file_processor_pb2.FileRequest(file_name=file_name, **request_params)

    # Faz o stream dos bytes nas mensagens seguintes
    with open(file_path, 'rb') as f:
        while True:
            chunk = f.read(1024)
            if not chunk:
                break
            yield file_processor_pb2.FileRequest(
                file_content=file_processor_pb2.FileChunk(content=chunk)
            )

def save_response_stream(response_stream, output_path):
    with open(output_path, 'wb') as f:
        for response in response_stream:
            if response.HasField('file_content'):
                f.write(response.file_content.content)
    return True

# ================================================================================
# CLASSE DO CLIENTE gRPC
# ================================================================================

class FileProcessorClient:
    def __init__(self, target='servidor-grpc:50051'):
        # Canal configurado para ignorar proxy e usar texto plano (sem SSL)
        self.channel = grpc.insecure_channel(
            target,
            options=[('grpc.enable_http_proxy', 0)]
        )
        self.stub = file_processor_pb2_grpc.FileProcessorServiceStub(self.channel)

    # ================================================================================
```

```
    # IMPLEMENTAÇÃO SUA
    # ============================================================================

    def compress_pdf(self, input_path, output_path):
        """
        TODO: Implementar a lógica para invocar a compressão de PDF no servidor.
        1. Crie o dicionário de parâmetros 'params' indicando a requisição
CompressPDFRequest.
        2. Chame a função auxiliar 'generate_requests(input_path, params)' para criar o
iterador.
        3. Envie o iterador para o método correto do 'self.stub' e guarde a resposta de
stream.
        4. Chame a função 'save_response_stream()' para salvar o arquivo final e retorne
o seu resultado.
        """
        pass # Remova o pass e escreva sua implementação aqui

    # ============================================================================
    # MÉTODOS IGNORADOS PARA ESTA ENTREGA
    # ============================================================================

    def convert_to_txt(self, input_path, output_path):
        # Não é necessário implementar para esta atividade.
        pass

    def convert_image(self, input_path, output_path, out_fmt):
        # Não é necessário implementar para esta atividade.
        pass

    def resize_image(self, input_path, output_path, w, h):
        # Não é necessário implementar para esta atividade.
        pass

# ============================================================================
# ÁREA DE TESTE (PODE ALTERAR OS CAMINHOS DOS ARQUIVOS)
# ============================================================================
if __name__ == '__main__':
    # Conecta no servidor usando as definições do compose
    cliente = FileProcessorClient(target='host.docker.internal:50051')
    # O padrao seria algo tipo
    #cliente = FileProcessorClient(target='servidor-grpc:50051')
    #mas usamos o host.... para contornar problemas de conexao (timeout) do codespaces
    #para teste no host do codespaces
    #cliente = FileProcessorClient(target='localhost:50051')

    print("Iniciando compressão...")

    # IMPORTANTE: O arquivo de origem precisa existir na pasta indicada.
    # Recomendado colocar uma cópia na mesma pasta deste script (ex: 'aula10.pdf')
    try:
        cliente.compress_pdf('aula10.pdf', 'comprimido.pdf')
        print("Compressão finalizada! Verifique o arquivo comprimido.pdf na sua pasta.")
    except Exception as e:
        print(f"Erro durante a execução: {e}")
```

Observações importantes:

- **Instalação de Dependências:** Vocês precisarão instalar as bibliotecas gRPC para C++ e Python, o compilador protobuf (protoc), Ghostscript e ImageMagick em seus ambientes de desenvolvimento.
- **Geração de Código gRPC**: Após definir o arquivo .proto, precisarão gerar o código gRPC em C++ e Python utilizando o compilador protoc com os plugins apropriados.
- **Tratamento de Erros:** A atividade deve enfatizar a importância do tratamento de erros tanto no servidor quanto nos clientes. Os exemplos fornecidos incluem um tratamento básico de erros, mas podem ser expandidos para cenários mais complexos.
- **Segurança:** Em um cenário real, seria crucial considerar a segurança, especialmente ao executar comandos de sistema e lidar com arquivos enviados por clientes. Para fins didáticos, a segurança pode ser abordada de forma mais superficial, mas é importante estas preocupações.
- **Utilização de Containers**
    - Essa é uma etapa extra neste projeto e deverá ser desenvolvida por cada grupo.
    - Todas as decisões de projeto para esta etapa deverão estar completamente documentadas, com bom detalhamento.

Solução completa (Conceitual):

A "solução completa" para esta atividade seria um projeto bem organizado com os seguintes componentes:

- **Diretório Raiz do Projeto:**
    - proto/: Contém o arquivo file_processor.proto.
    - server_cpp/: Contém o código fonte do servidor C++ (server.cc, file_processor_service_impl.cc, CMakeLists.txt, etc.).
    - client_python/: Contém o código fonte do cliente Python (client.py, , etc.).
    - scripts/: Scripts úteis para compilação, execução e testes (opcional).
    - README.md: Instruções para compilar, executar e testar a atividade.
- **Arquivos Chave:**
    - proto/file_processor.proto: Definição dos serviços e mensagens gRPC. o server_cpp/server.cc: Ponto de entrada do servidor gRPC em C++.
    - server_cpp/file_processor_service_impl.cc: Implementação da classe FileProcessorServiceImpl.
    - client_python/client.py: Cliente gRPC em Python.
    - server.log: Arquivo de log gerado pelo servidor.

Fluxo de execução típico:

1. **Compilar Protobuf:** Utilizar protoc para gerar código gRPC em C++ e Python a partir do arquivo file_processor.proto.
2. **Compilar Servidor C++:** Utilizar CMake (ou outro sistema de build) para compilar o servidor C++ e gerar o executável do servidor.
3. **Instalar Dependências Python:** Utilizar pip install -r requirements.txt (se houver) para instalar as dependências do cliente Python.
4. **Executar Servidor:** Iniciar o executável do servidor C++.
5. **Executar Cliente:** Executar o cliente Python, fornecendo os arquivos de entrada e os parâmetros desejados.
6. **Verificar Resultados:** Verificar os arquivos de saída gerados pelos clientes e o arquivo de log do servidor para confirmar o correto funcionamento do sistema.

Dicas para a realização da tarefa

Dockerfile do client

```
FROM python:3.10-slim

# Instalar dependências básicas do sistema
RUN apt-get update && apt-get install -y build-essential && rm -rf /var/lib/apt/lists/*

# Copiar os requisitos do Python
COPY client_python/requirements.txt /tmp/
RUN pip install --no-cache-dir -r /tmp/requirements.txt

WORKDIR /workspace
```

Dockerfile do server

```
FROM ubuntu:22.04

# Evitar prompts interativos durante a instalação do apt
ENV DEBIAN_FRONTEND=noninteractive

# Atualizar pacotes e instalar todas as dependências do servidor
RUN apt-get update && apt-get install -y \
    build-essential \
    cmake \
    git \
    pkg-config \
    ghostscript \
    poppler-utils \
    imagemagick \
    libgrpc++-dev \
    libprotobuf-dev \
    protobuf-compiler-grpc \
    protobuf-compiler \
    && rm -rf /var/lib/apt/lists/*

# Configuração de permissões do ImageMagick para leitura/escrita de PDFs (se necessário
para conversões extras)
RUN sed -i 's/rights="none" pattern="PDF"/rights="read|write" pattern="PDF"/g'
/etc/ImageMagick-6/policy.xml || true

WORKDIR /workspace
```

CMakeLists.txt (para rodar local no codespaces)

```
#cd server_cpp
#mkdir build
#cd build
#cmake ..
#make
#./server

cmake_minimum_required(VERSION 3.15)
project(FileProcessorServer CXX)

# O gRPC requer C++17 nas versões mais recentes
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Localizar os pacotes do gRPC e Protobuf no sistema
find_package(gRPC CONFIG REQUIRED)
find_package(Protobuf REQUIRED)

# Diretório onde está o arquivo proto
set(PROTO_DIR "${CMAKE_CURRENT_SOURCE_DIR}/../proto")
set(PROTO_FILE "${PROTO_DIR}/file_processor.proto")

# Variáveis com os nomes dos arquivos gerados automaticamente
set(PROTO_SRC "${CMAKE_CURRENT_BINARY_DIR}/file_processor.pb.cc")
set(PROTO_HDR "${CMAKE_CURRENT_BINARY_DIR}/file_processor.pb.h")
set(GRPC_SRC "${CMAKE_CURRENT_BINARY_DIR}/file_processor.grpc.pb.cc")
set(GRPC_HDR "${CMAKE_CURRENT_BINARY_DIR}/file_processor.grpc.pb.h")

# Comando customizado para compilar o .proto e gerar o código C++
add_custom_command(
   OUTPUT ${PROTO_SRC} ${PROTO_HDR} ${GRPC_SRC} ${GRPC_HDR}
   COMMAND protobuf::protoc
   ARGS --grpc_out="${CMAKE_CURRENT_BINARY_DIR}"
      --cpp_out="${CMAKE_CURRENT_BINARY_DIR}"
      -I "${PROTO_DIR}"
      --plugin=protoc-gen-grpc=$<TARGET_FILE:gRPC::grpc_cpp_plugin>
      "${PROTO_FILE}"
   DEPENDS "${PROTO_FILE}"
)

# Incluir o diretório onde os arquivos de cabeçalho (.h) foram gerados
include_directories("${CMAKE_CURRENT_BINARY_DIR}")

# Declarar o executável do servidor, incluindo os arquivos gerados pelo protoc
add_executable(server
   server.cc
   ${PROTO_SRC}
   ${GRPC_SRC}
)

# Vincular (link) as bibliotecas do gRPC e Protobuf ao executável
```

```
target_link_libraries(server
    gRPC::grpc++
    gRPC::grpc
    protobuf::libprotobuf
)
```

CMakeLists.txt (para rodar com containers no codespaces)

```
cmake_minimum_required(VERSION 3.15)
project(FileProcessorServer CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Usa o pkg-config do Ubuntu para localizar o gRPC e Protobuf
find_package(PkgConfig REQUIRED)
pkg_check_modules(GRPC REQUIRED grpc++)
pkg_check_modules(PROTOBUF REQUIRED protobuf)

set(PROTO_DIR "${CMAKE_CURRENT_SOURCE_DIR}/../proto")
set(PROTO_FILE "${PROTO_DIR}/file_processor.proto")

set(PROTO_SRC "${CMAKE_CURRENT_BINARY_DIR}/file_processor.pb.cc")
set(PROTO_HDR "${CMAKE_CURRENT_BINARY_DIR}/file_processor.pb.h")
set(GRPC_SRC "${CMAKE_CURRENT_BINARY_DIR}/file_processor.grpc.pb.cc")
set(GRPC_HDR "${CMAKE_CURRENT_BINARY_DIR}/file_processor.grpc.pb.h")

# No Ubuntu, o apt-get instala o compilador e o plugin nestes caminhos fixos
set(PROTOC_COMPILER "/usr/bin/protoc")
set(GRPC_CPP_PLUGIN "/usr/bin/grpc_cpp_plugin")

add_custom_command(
    OUTPUT ${PROTO_SRC} ${PROTO_HDR} ${GRPC_SRC} ${GRPC_HDR}
    COMMAND ${PROTOC_COMPILER}
    ARGS --grpc_out="${CMAKE_CURRENT_BINARY_DIR}"
         --cpp_out="${CMAKE_CURRENT_BINARY_DIR}"
         -I "${PROTO_DIR}"
         --plugin=protoc-gen-grpc=${GRPC_CPP_PLUGIN}
         "${PROTO_FILE}"
    DEPENDS "${PROTO_FILE}"
)

include_directories("${CMAKE_CURRENT_BINARY_DIR}")

add_executable(server
    server.cc
    ${PROTO_SRC}
    ${GRPC_SRC}
)

# Link direto com as bibliotecas do sistema
target_link_libraries(server
    grpc++
```

```
    grpc
    protobuf
)
```

Dependências do cliente

Arquivo requirements.txt no cliente

```
grpcio>=1.50.0
grpcio-tools>=1.50.0
pytest>=7.0.0
```

docker-compose.yml

```yaml
services:
  servidor-grpc:
    build:
      context: .
      dockerfile: server_cpp/Dockerfile
    ports:
      - "50051:50051"
    volumes:
      - .:/workspace
    command: sleep infinity

  cliente-python:
    build:
      context: .
      dockerfile: client_python/Dockerfile
    volumes:
      - .:/workspace
    depends_on:
      - servidor-grpc
    extra_hosts:
      - "host.docker.internal:host-gateway"
    command: sleep infinity
```

Roteiro de alguns comandos:

#roteiro sem docker

#terminal 1
sudo apt-get update
sudo apt-get install build-essential cmake ghostscript poppler-utils imagemagick libgrpc++-dev libprotobuf-dev protobuf-compiler-grpc protobuf-compiler python3 python3-pip
cd server_cpp
mkdir build
cd build
cmake ..
make
./server

#terminal 2
cd client_python
pip install grpcio grpcio-tools
python3 -m grpc_tools.protoc -I../proto --python_out=. --grpc_python_out=. ../proto/file_processor.proto
python3 client.py

#roteiro com docker
#na pasta raiz
docker-compose up -d --build
#terminal 1
docker exec -it arquivos-servidor-grpc-1 bash
cd server_cpp
cd build
rm -rf *
#altere o CMAKEFILE para o Ubuntu
cmake ..
make
./server

#terminal 2
docker exec -it arquivos-cliente-python-1 bash
cd client_python
python -m grpc_tools.protoc -I../proto --python_out=. --grpc_python_out=. ../proto/file_processor.proto
python3 client.py

#não se esqueça, sempre que fizer alterações nos códigos, atualize os containers
docker-compose down
docker-compose up -d --build